

Proyecto de Grado: IA al servicio de las comunidades

Modelo de predicción de enfermedades en Yucas

Datos tomados de: <https://www.kaggle.com/competitions/cassava-leaf-disease-classification/data>

Nombres: María Fernanda Izquierdo Aparicio & Ana María Ochoa Muñoz

Fecha: 9 de Marzo de 2025

1. Importe de librerías

```
In [3]: import json

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.image as mpimg

import os
import zipfile
import shutil
import seaborn as sns
import datetime
import cv2

import keras
from keras import models
from keras import layers

import random

import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras import Layer
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.preprocessing import image
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.preprocessing.image import img_to_array, load_img
from tensorflow.keras.callbacks import ReduceLROnPlateau
```

```

from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping, ReduceLROnPlateau
from tensorflow.keras.applications import EfficientNetB0
from tensorflow.keras.optimizers import Adam

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

from PIL import Image # Librería para abrir y manipular imágenes

# ignoring warnings
import warnings
warnings.simplefilter("ignore")

```

In []:

In []:

```

In [4]: #Medir_tiempo_ejecucion():
        inicio = datetime.datetime.now()
        print(f"Inicio: {inicio.strftime('%Y-%m-%d %H:%M:%S.%f')}")

```

Inicio: 2025-05-24 11:18:40.414334

2. Importe de Datos (Yuca)

```

In [38]: # Nombre del archivo JSON
        archivo_json = "label_num_to_disease_map.json"

        # Leer el archivo JSON
        try:
            with open(archivo_json, "r", encoding="utf-8") as file:
                datos = json.load(file)
            print("Contenido del JSON:", datos)
        except FileNotFoundError:
            print(f"El archivo '{archivo_json}' no se encontró.")
        except json.JSONDecodeError:
            print("Error al decodificar el JSON. Verifica su formato.")

```

Contenido del JSON: {'0': 'Cassava Bacterial Blight (CBB)', '1': 'Cassava Brown Streak Disease (CBSD)', '2': 'Cassava Green Mottle (CGM)', '3': 'Cassava Mosaic Disease (CMD)', '4': 'Healthy'}

Importe del archivo JSON que tiene el mapa de las etiquetas de numeros a las etiquetas descriptivas

Importe del archivo CSV que tiene el listado de los archivos de las imagenes con las etiquetas de números

```
In [42]: train=pd.read_csv('train.csv')
train.head()
```

```
Out[42]:
```

	image_id	label
0	1000015157.jpg	0
1	1000201771.jpg	3
2	100042118.jpg	1
3	1000723321.jpg	1
4	1000812911.jpg	3

```
In [44]: train['label']=train['label'].astype('str').replace(datos)
train.head()
```

```
Out[44]:
```

	image_id	label
0	1000015157.jpg	Cassava Bacterial Blight (CBB)
1	1000201771.jpg	Cassava Mosaic Disease (CMD)
2	100042118.jpg	Cassava Brown Streak Disease (CBSD)
3	1000723321.jpg	Cassava Brown Streak Disease (CBSD)
4	1000812911.jpg	Cassava Mosaic Disease (CMD)

Reemplazo en el dataframe de train las etiquetas de números por las etiquetas descriptivas

3. Análisis de Datos y Creación de conjuntos de train & test

Conteo de imágenes por categoría

```
In [49]: train.label.value_counts()
```

```
Out[49]:
```

label	
Cassava Mosaic Disease (CMD)	13158
Healthy	2577
Cassava Green Mottle (CGM)	2386
Cassava Brown Streak Disease (CBSD)	2189
Cassava Bacterial Blight (CBB)	1087
Name: count, dtype: int64	

Division aleatoria entre conjunto de train y test usando una division 80% - 20%

```
In [52]: train, test= train_test_split( train, test_size=0.2, random_state=42, shuffle=True)
train.shape, test.shape
```

```
Out[52]: ((17117, 2), (4280, 2))
```

Conteo de imágenes por en el conjunto train por categoría

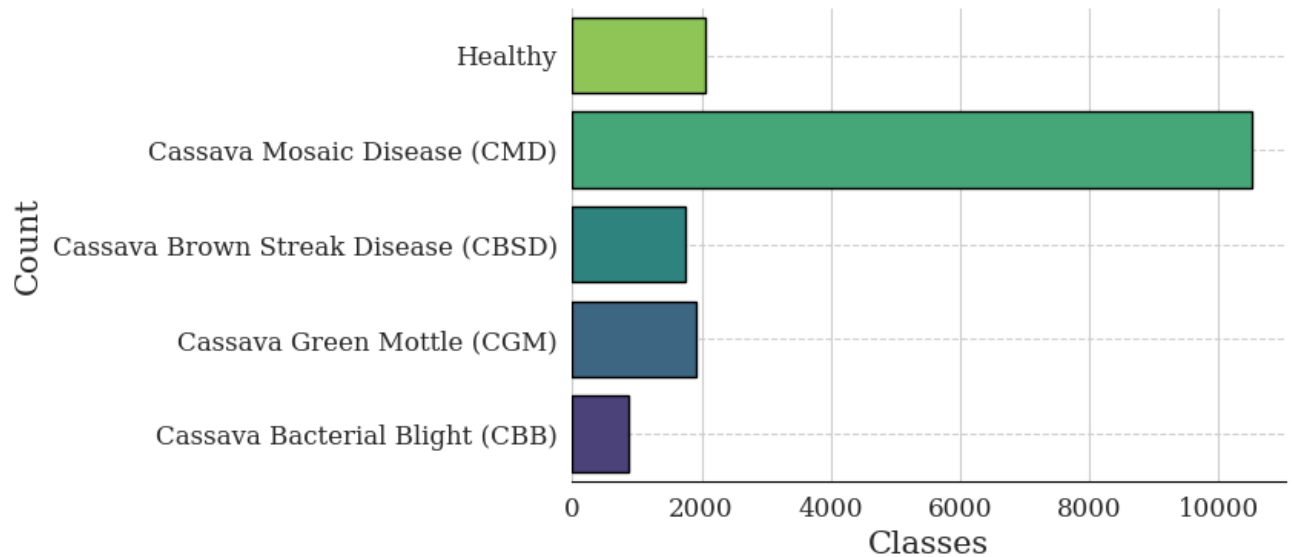
```
In [55]: train.label.value_counts()
```

```
Out[55]: label
Cassava Mosaic Disease (CMD)      10526
Healthy                          2061
Cassava Green Mottle (CGM)       1909
Cassava Brown Streak Disease (CBSD) 1751
Cassava Bacterial Blight (CBB)    870
Name: count, dtype: int64
```

```
In [57]: sns.set_style("whitegrid")
fig, ax = plt.subplots(figsize = (6, 4))

for i in ['top', 'right', 'left']:
    ax.spines[i].set_visible(False)
ax.spines['bottom'].set_color('black')

sns.countplot(train.label, edgecolor = 'black',
               palette = reversed(sns.color_palette("viridis", 5)))
plt.xlabel('Classes', fontfamily = 'serif', size = 15)
plt.ylabel('Count', fontfamily = 'serif', size = 15)
plt.xticks(fontfamily = 'serif', size = 12)
plt.yticks(fontfamily = 'serif', size = 12)
ax.grid(axis = 'y', linestyle = '--', alpha = 0.9)
plt.show()
```

3.1. Muestra de Imagenes

Plantas Healthy

```
In [66]: # Seleccionar 3 muestras de la clase "Healthy"
sample = train[train.label == "Healthy"].sample(3)

# Tamaño de la figura
plt.figure(figsize=(15, 5))

# Carpeta donde están las imágenes
image_folder = "Datos por Planta/Yuca"

for ind, (image_id, label) in enumerate(zip(sample.image_id, sample.label)):
    plt.subplot(1, 3, ind + 1)

    # Construir la ruta completa de la imagen
    image_path = os.path.join(image_folder, image_id)

    # Cargar la imagen correctamente
    img = cv2.imread(image_path)

    # Verificar si la imagen se cargó correctamente
    if img is None:
        print(f"Error al cargar la imagen: {image_path}")
        continue

    # Convertir la imagen de BGR a RGB
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    # Mostrar la imagen
```

```
plt.imshow(img)
plt.axis("off")

plt.show()
```



Plantas con Cassava Mosaic Disease (CMD)

```
In [71]: # Seleccionar 3 muestras de la clase "Cassava Mosaic Disease (CMD)"
sample = train[train.label == "Cassava Mosaic Disease (CMD)"].sample(3)

# Tamaño de la figura
plt.figure(figsize=(15, 5))

# Carpeta donde están las imágenes
image_folder = "Datos por Planta/Yuca"

for ind, (image_id, label) in enumerate(zip(sample.image_id, sample.label)):
    plt.subplot(1, 3, ind + 1)

    # Construir la ruta completa de la imagen
    image_path = os.path.join(image_folder, image_id)

    # Cargar la imagen correctamente
    img = cv2.imread(image_path)

    # Verificar si la imagen se cargó correctamente
    if img is None:
        print(f"Error al cargar la imagen: {image_path}")
        continue

    # Convertir la imagen de BGR a RGB
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    # Mostrar la imagen
    plt.imshow(img)
    plt.axis("off")

plt.show()
```



Plantas con Cassava Brown Streak Disease (CBSD)

```
In [74]: # Seleccionar 3 muestras de la clase "Cassava Brown Streak Disease (CBSD)"
sample = train[train.label == "Cassava Brown Streak Disease (CBSD)"].sample(

# Tamaño de la figura
plt.figure(figsize=(15, 5))

# Carpeta donde están las imágenes
image_folder = "Datos por Planta/Yuca"

for ind, (image_id, label) in enumerate(zip(sample.image_id, sample.label)):
    plt.subplot(1, 3, ind + 1)

    # Construir la ruta completa de la imagen
    image_path = os.path.join(image_folder, image_id)

    # Cargar la imagen correctamente
    img = cv2.imread(image_path)

    # Verificar si la imagen se cargó correctamente
    if img is None:
        print(f"Error al cargar la imagen: {image_path}")
        continue

    # Convertir la imagen de BGR a RGB
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    # Mostrar la imagen
    plt.imshow(img)
    plt.axis("off")

plt.show()
```



Plantas con Cassava Green Mottle (CGM)

```
In [76]: # Seleccionar 3 muestras de la clase "Cassava Green Mottle (CGM)"
sample = train[train.label == "Cassava Green Mottle (CGM)"].sample(3)

# Tamaño de la figura
plt.figure(figsize=(15, 5))

# Carpeta donde están las imágenes
image_folder = "Datos por Planta/Yuca"

for ind, (image_id, label) in enumerate(zip(sample.image_id, sample.label)):
    plt.subplot(1, 3, ind + 1)

    # Construir la ruta completa de la imagen
    image_path = os.path.join(image_folder, image_id)

    # Cargar la imagen correctamente
    img = cv2.imread(image_path)

    # Verificar si la imagen se cargó correctamente
    if img is None:
        print(f"Error al cargar la imagen: {image_path}")
        continue

    # Convertir la imagen de BGR a RGB
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    # Mostrar la imagen
    plt.imshow(img)
    plt.axis("off")

plt.show()
```



Plantas con Cassava Bacterial Blight (CBB)

```
In [78]: # Seleccionar 3 muestras de la clase "Cassava Bacterial Blight (CBB)"
sample = train[train.label == "Cassava Bacterial Blight (CBB)"].sample(3)

# Tamaño de la figura
plt.figure(figsize=(15, 5))

# Carpeta donde están las imágenes
image_folder = "Datos por Planta/Yuca"

for ind, (image_id, label) in enumerate(zip(sample.image_id, sample.label)):
    plt.subplot(1, 3, ind + 1)

    # Construir la ruta completa de la imagen
    image_path = os.path.join(image_folder, image_id)

    # Cargar la imagen correctamente
    img = cv2.imread(image_path)

    # Verificar si la imagen se cargó correctamente
    if img is None:
        print(f"Error al cargar la imagen: {image_path}")
        continue

    # Convertir la imagen de BGR a RGB
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    # Mostrar la imagen
    plt.imshow(img)
    plt.axis("off")

plt.show()
```




Obtener el tamaño de las imágenes

```
In [81]: # Definir la carpeta con las imágenes
image_folder = "Datos por Planta/Yuca"

# Función para obtener el tamaño de una imagen
def get_image_size(image_path):
    try:
        with Image.open(image_path) as img:
            return img.size # Devuelve (ancho, alto)
    except Exception as e:
        print(f"Error con la imagen {image_path}: {e}")
        return None, None

# Obtener el tamaño de cada imagen y agregarlo al DataFrame
widths = []
heights = []

for img_name in train["image_id"]: # Asegúrate de que la columna se llama "
    img_path = os.path.join(image_folder, img_name)
    width, height = get_image_size(img_path)
    widths.append(width)
    heights.append(height)

train["width"] = widths
train["height"] = heights

# Mostrar las primeras filas con los nuevos datos
print(train.head())
```

	image_id	label	width	height
1715	1300823152.jpg	Healthy	800	600
11830	3116613979.jpg	Cassava Mosaic Disease (CMD)	800	600
8621	2530012971.jpg	Cassava Mosaic Disease (CMD)	800	600
3849	1686186154.jpg	Cassava Mosaic Disease (CMD)	800	600
12924	3312736716.jpg	Cassava Brown Streak Disease (CBSD)	800	600

4. Preparación del Modelo

```
In [83]: # Hiperparámetros principales
BATCH_SIZE = 8
STEPS_PER_EPOCH = len(train)*0.8 / BATCH_SIZE
VALIDATION_STEPS = len(train)*0.2 / BATCH_SIZE
EPOCHS = 8
TARGET_SIZE = 150
```

```
In [87]: train.label = train.label.astype('str')

train_datagen = ImageDataGenerator(validation_split = 0.2,
                                   preprocessing_function = None,
                                   rotation_range = 45,
                                   zoom_range = 0.2,
                                   horizontal_flip = True,
                                   vertical_flip = True,
                                   fill_mode = 'nearest',
                                   shear_range = 0.1,
                                   height_shift_range = 0.1,
                                   width_shift_range = 0.1)

train_generator = train_datagen.flow_from_dataframe(train,
                                                    directory = "Datos por Planta/Yuca",
                                                    subset = "training",
                                                    x_col = "image_id",
                                                    y_col = "label",
                                                    target_size = (TARGET_SIZE, TARGET_SIZE),
                                                    batch_size = BATCH_SIZE,
                                                    class_mode = "categorical")

validation_datagen = ImageDataGenerator(validation_split = 0.2)

validation_generator = validation_datagen.flow_from_dataframe(train,
                                                             directory = "Datos por Planta/Yuca",
                                                             subset = "validation",
                                                             x_col = "image_id",
                                                             y_col = "label",
                                                             target_size = (TARGET_SIZE, TARGET_SIZE),
                                                             batch_size = BATCH_SIZE,
                                                             class_mode = "categorical")
```

Found 13694 validated image filenames belonging to 5 classes.

Found 3423 validated image filenames belonging to 5 classes.

Imágen Aleatoria antes de Augmentation

```
In [89]: img_path = os.path.join("Datos por Planta/Yuca", train.image_id[20])
img = image.load_img(img_path, target_size = (TARGET_SIZE, TARGET_SIZE))
img_tensor = image.img_to_array(img)
```

```
img_tensor = np.expand_dims(img_tensor, axis = 0)
img_tensor /= 255.

plt.imshow(img_tensor[0])
plt.axis('off')
plt.show()
```



```
In [91]: generator = train_datagen.flow_from_dataframe(train.iloc[20:21],
    directory = "Datos por Planta/Yuca",
    x_col = "image_id",
    y_col = "label",
    target_size = (TARGET_SIZE, TARGET_SIZE),
    batch_size = BATCH_SIZE,
    class_mode = "categorical")

aug_images = [generator[0][0][0]/255 for i in range(10)]
fig, axes = plt.subplots(2, 5, figsize = (20, 10))
axes = axes.flatten()
for img, ax in zip(aug_images, axes):
    ax.imshow(img)
    ax.axis('off')
plt.tight_layout()
plt.show()
```

Found 1 validated image filenames belonging to 1 classes.



5. Construcción del Modelo

5.1. Clasificador

```
In [93]: # Arquitectura del modelo CNN
model = models.Sequential()

# Capa 1: Convolucional y MaxPooling
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(150, 150, 3)))
model.add(layers.BatchNormalization())
model.add(layers.MaxPooling2D(2, 2))

# Capa 2: Convolucional y MaxPooling
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.BatchNormalization())
model.add(layers.MaxPooling2D(2, 2))

# Capa 3: Convolucional y MaxPooling
model.add(layers.Conv2D(128, (3, 3), activation='relu'))
model.add(layers.BatchNormalization())
model.add(layers.MaxPooling2D(2, 2))

# Capa 4: Convolucional y MaxPooling
model.add(layers.Conv2D(128, (3, 3), activation='relu'))
model.add(layers.BatchNormalization())
model.add(layers.MaxPooling2D(2, 2))

# Aplanamiento de la capa convolucional
model.add(layers.Flatten())

# Clasificador
```

```
model.add(layers.Dense(512, activation='relu'))
model.add(layers.Dropout(0.5)) # Dropout para evitar el overfitting

# Capa de salida
model.add(layers.Dense(5, activation='softmax'))
```

5.2. Compilación del modelo

```
In [95]: model.compile(
    loss='categorical_crossentropy', # Cambiado a 'categorical_crossentropy'
    optimizer='adam',
    metrics=['accuracy']
)
```

```
In [97]: model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Par
conv2d (Conv2D)	(None, 148, 148, 32)	
batch_normalization (BatchNormalization)	(None, 148, 148, 32)	
max_pooling2d (MaxPooling2D)	(None, 74, 74, 32)	
conv2d_1 (Conv2D)	(None, 72, 72, 64)	18
batch_normalization_1 (BatchNormalization)	(None, 72, 72, 64)	
max_pooling2d_1 (MaxPooling2D)	(None, 36, 36, 64)	
conv2d_2 (Conv2D)	(None, 34, 34, 128)	73
batch_normalization_2 (BatchNormalization)	(None, 34, 34, 128)	
max_pooling2d_2 (MaxPooling2D)	(None, 17, 17, 128)	
conv2d_3 (Conv2D)	(None, 15, 15, 128)	147
batch_normalization_3 (BatchNormalization)	(None, 15, 15, 128)	
max_pooling2d_3 (MaxPooling2D)	(None, 7, 7, 128)	
flatten (Flatten)	(None, 6272)	
dense (Dense)	(None, 512)	3,211
dropout (Dropout)	(None, 512)	
dense_1 (Dense)	(None, 5)	2

Total params: 3,456,581 (13.19 MB)

Trainable params: 3,455,877 (13.18 MB)

Non-trainable params: 704 (2.75 KB)

6. Ejecución del modelo

```
In [99]: early_stopping = EarlyStopping(
    monitor='val_loss', # Monitor the validation loss
    patience=5, # Number of epochs with no improvement after which training
    restore_best_weights=True # Restore the model weights from the epoch wi
```

```
)

from tensorflow.keras.callbacks import ReduceLRonPlateau

reduce_lr = ReduceLRonPlateau(
    monitor='val_loss',
    factor=0.2, # Reduce learning rate by a factor of 0.2
    patience=3, # Number of epochs with no improvement after which learning
    min_lr=1e-5 # Minimum learning rate
)
```

```
In [101... history = model.fit(
    train_generator,
    epochs=EPOCHS,
    validation_data=validation_generator,
    callbacks=[early_stopping, reduce_lr],
    verbose=1
)
```

```
Epoch 1/8
1712/1712 _____ 227s 132ms/step - accuracy: 0.5417 - loss: 2.
6307 - val_accuracy: 0.6115 - val_loss: 1.0787 - learning_rate: 0.0010
Epoch 2/8
1712/1712 _____ 231s 135ms/step - accuracy: 0.6118 - loss: 1.
1306 - val_accuracy: 0.6164 - val_loss: 1.1064 - learning_rate: 0.0010
Epoch 3/8
1712/1712 _____ 237s 138ms/step - accuracy: 0.6081 - loss: 1.
1208 - val_accuracy: 0.6345 - val_loss: 1.0657 - learning_rate: 0.0010
Epoch 4/8
1712/1712 _____ 241s 141ms/step - accuracy: 0.6183 - loss: 1.
0826 - val_accuracy: 0.6372 - val_loss: 0.9761 - learning_rate: 0.0010
Epoch 5/8
1712/1712 _____ 249s 146ms/step - accuracy: 0.6376 - loss: 1.
0409 - val_accuracy: 0.6567 - val_loss: 0.9089 - learning_rate: 0.0010
Epoch 6/8
1712/1712 _____ 251s 147ms/step - accuracy: 0.6253 - loss: 1.
0229 - val_accuracy: 0.6637 - val_loss: 0.8893 - learning_rate: 0.0010
Epoch 7/8
1712/1712 _____ 279s 163ms/step - accuracy: 0.6415 - loss: 0.
9821 - val_accuracy: 0.6731 - val_loss: 0.9288 - learning_rate: 0.0010
Epoch 8/8
1712/1712 _____ 265s 155ms/step - accuracy: 0.6523 - loss: 0.
9473 - val_accuracy: 0.6582 - val_loss: 0.9063 - learning_rate: 0.0010
```

6.1. Resultados del Modelo

```
In [102... # Acceder a los datos de pérdida y precisión
loss = history.history['loss']
val_loss = history.history['val_loss']
```

```

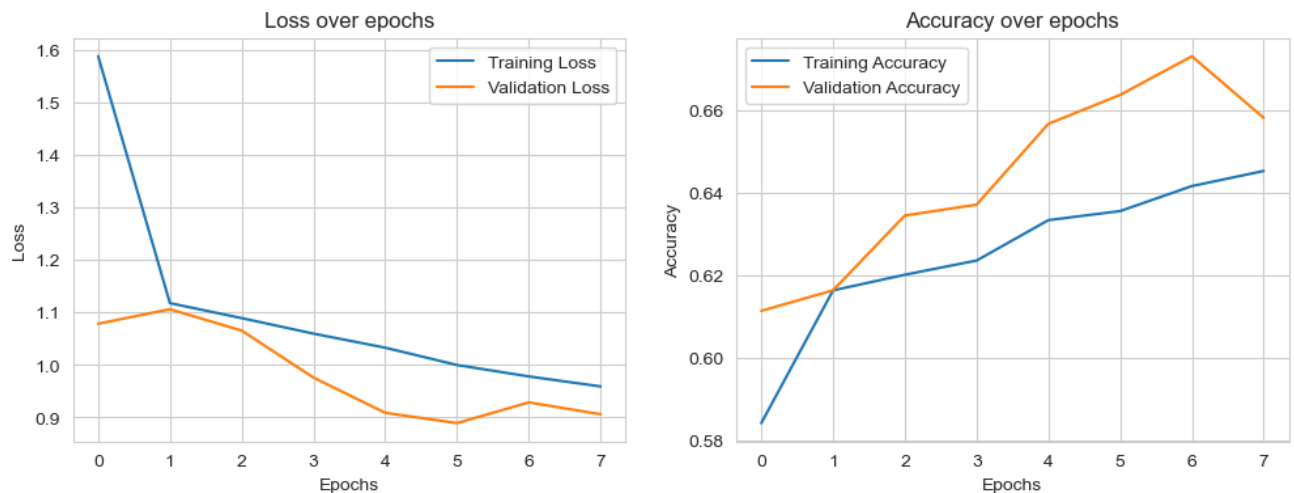
accuracy = history.history['accuracy']
val_accuracy = history.history['val_accuracy']

# Graficar la pérdida del modelo
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Loss over epochs')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()

# Graficar la precisión del modelo
plt.subplot(1, 2, 2)
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Accuracy over epochs')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()

```

Out[102...] <matplotlib.legend.Legend at 0x297b337a0>



```

In [103...] epochs = [1, 2, 3, 4, 5, 6, 7, 8]
data = {
    'Epoch': epochs,
    'Trainning Loss': loss,
    'Validation Loss': val_loss,
    'Training Accuracy': accuracy,
    'Validation Accuracy': val_accuracy
}
df = pd.DataFrame(data)

# Imprimir la tabla
print(df)

```

	Epoch	Trainning Loss	Validation Loss	Training Accuracy \
0	1	1.588456	1.078723	0.584270
1	2	1.117934	1.106353	0.616401
2	3	1.089402	1.065661	0.620199
3	4	1.059855	0.976066	0.623631
4	5	1.033046	0.908919	0.633416
5	6	1.000026	0.889328	0.635607
6	7	0.978254	0.928768	0.641668
7	8	0.959225	0.906283	0.645319

	Validation Accuracy
0	0.611452
1	0.616418
2	0.634531
3	0.637160
4	0.656734
5	0.663745
6	0.673094
7	0.658195

```
In [121... fin = datetime.datetime.now()
print(f"Fin: {fin.strftime('%Y-%m-%d %H:%M:%S.%f')}")
```

Fin: 2025-05-24 11:57:14.201757

```
In [123... duracion = fin - inicio
print(f"Duración: {duracion}")
```

Duración: 0:38:33.787423

7. Predicción

```
In [128... #validación con datos de test
test_generator = train_datagen.flow_from_dataframe(test,
                                                    directory = "Datos por Planta/Yuca",
                                                    x_col = "image_id",
                                                    y_col = "label",
                                                    target_size = (TARGET_SIZE, TARGET_SIZE),
                                                    batch_size = 1,
                                                    shuffle=False,
                                                    class_mode = "categorical")
```

Found 4280 validated image filenames belonging to 5 classes.

```
In [130... # Evaluar sobre el set de test
test_loss, test_accuracy = model.evaluate(test_generator, steps=len(test_gen

print(f'🔍 Pérdida en test: {test_loss:.4f}')
print(f'✅ Precisión en test: {test_accuracy:.2%}')
```


4280/4280 ————— 32s 8ms/step – accuracy: 0.6285 – loss: 0.968

0

🔍 Pérdida en test: 0.9579

✅ Precisión en test: 63.43%

```
In [131... # 1. Predecir clases sobre el test set
y_pred_probs = model.predict(test_generator)

# Convertir probabilidades a etiquetas (argmax)
y_pred = np.argmax(y_pred_probs, axis=1)

# 2. Obtener etiquetas verdaderas
y_true = test_generator.classes
```

4280/4280 ————— 34s 8ms/step

```
In [132... import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns

# 1. Predecir clases sobre el test set
# Obtener las predicciones del modelo
# y_pred_probs = model.predict(test_generator, steps=len(test_generator), ver

# Convertir probabilidades a etiquetas (argmax)
# _pred = np.argmax(y_pred_probs, axis=1)

# 2. Obtener etiquetas verdaderas
# Las verdaderas etiquetas están en test_generator.classes
# y_true = test_generator.classes

# 3. Obtener los nombres de las clases
class_labels = list(test_generator.class_indices.keys())

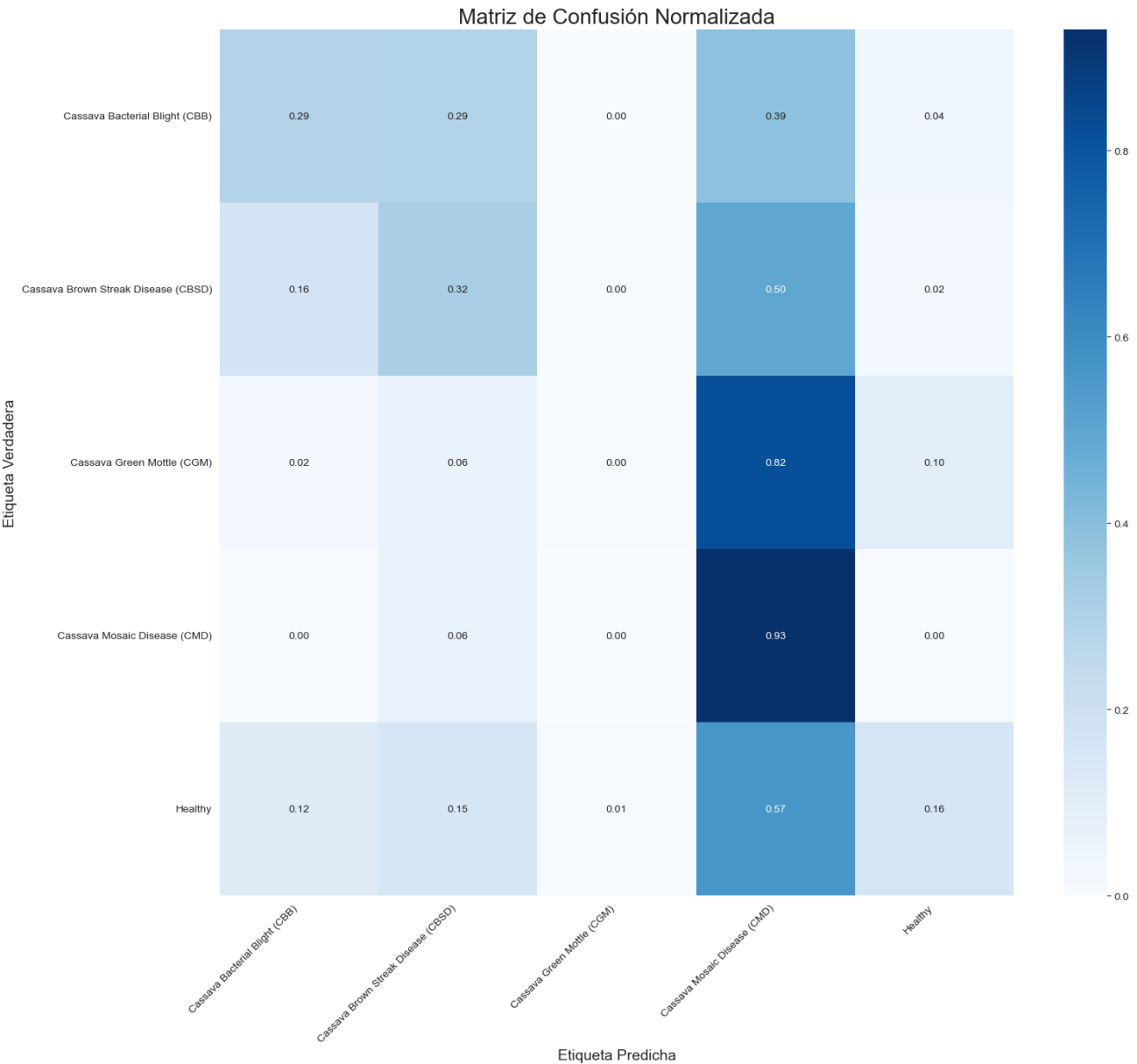
# 4. Calcular la matriz de confusión
cm = confusion_matrix(y_true, y_pred)

# 5. Normalizar la matriz de confusión (opcional para mejor visualización)
cm_normalized = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]

# 6. Graficar la matriz de confusión
plt.figure(figsize=(16, 14))
sns.heatmap(cm_normalized, annot=True, fmt='.2f', cmap='Blues', xticklabels=
plt.title('Matriz de Confusión Normalizada', fontsize=20)
plt.ylabel('Etiqueta Verdadera', fontsize=14)
plt.xlabel('Etiqueta Predicha', fontsize=14)
plt.xticks(rotation=45, ha='right')
plt.yticks(rotation=0)
```

```
plt.tight_layout()
plt.show()

# 7. Reporte de clasificación (precisión, recall, f1-score por clase)
print('\nReporte de Clasificación:')
print(classification_report(y_true, y_pred, target_names=class_labels, digit
```



Reporte de Clasificación:

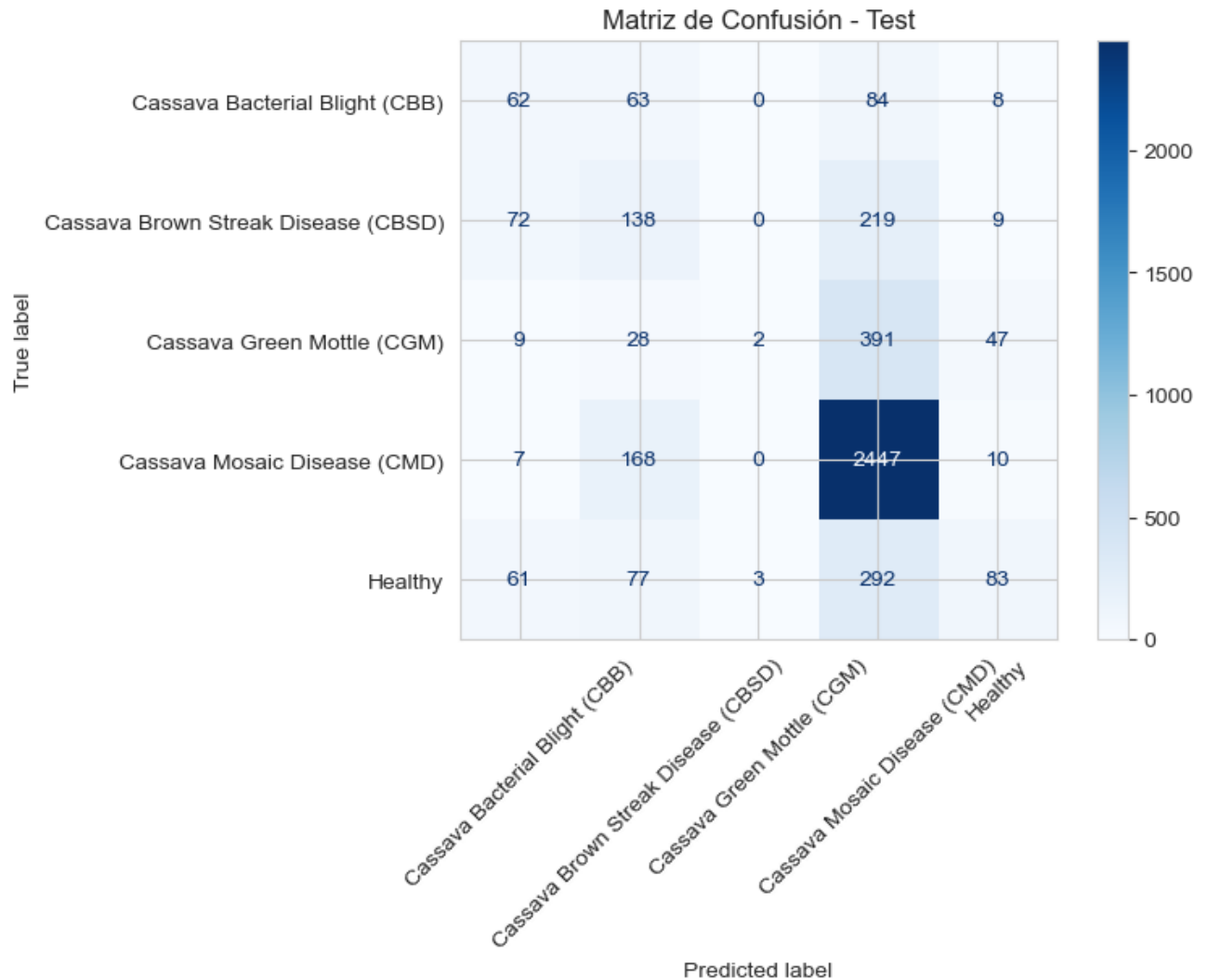
	precision	recall	f1-score	support
Cassava Bacterial Blight (CBB)	0.2938	0.2857	0.2897	217
Cassava Brown Streak Disease (CBSD)	0.2911	0.3151	0.3026	438
Cassava Green Mottle (CGM)	0.4000	0.0042	0.0083	477
Cassava Mosaic Disease (CMD)	0.7128	0.9297	0.8069	2632
Healthy	0.5287	0.1609	0.2467	516
accuracy			0.6383	4280
macro avg	0.4453	0.3391	0.3308	4280
weighted avg	0.5913	0.6383	0.5725	4280

```
In [133... accuracy = accuracy_score(y_true, y_pred)
print(f"Test Accuracy: {accuracy:.4f}")
```

Test Accuracy: 0.6383

```
In [134... # -----
# 2. Matriz de Confusión
# -----
from sklearn.metrics import confusion_matrix, accuracy_score, ConfusionMatrixDisplay

cm = confusion_matrix(y_true, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=test_generator.classes_)
disp.plot(cmap="Blues", xticks_rotation=45)
plt.title("Matriz de Confusión - Test")
plt.show()
```



```
In [135... # Función para normalizar imágenes solo para visualización
def normalize_for_display(img):
    img_min = img.min()
    img_max = img.max()
    if img_max > 1.0 or img_min < 0.0:
        img = (img - img_min) / (img_max - img_min + 1e-7)
    return img

# Obtener nombres de clases
class_indices = test_generator.class_indices
class_names = list(class_indices.keys())

# Reiniciar el generador para empezar desde el principio
test_generator.reset()
x_batch, y_true_batch = next(test_generator)

# Obtener predicciones del modelo
y_pred_probs_batch = model.predict(x_batch)
y_pred_batch = np.argmax(y_pred_probs_batch, axis=1)
```

```

y_true_batch = np.argmax(y_true_batch, axis=1)

# Número de imágenes a mostrar (máximo el tamaño del batch)
num_samples = x_batch.shape[0]

# Mostrar imágenes con etiquetas
plt.figure(figsize=(15, 8))
for i in range(num_samples):
    plt.subplot(3, 4, i + 1)
    img_to_show = normalize_for_display(x_batch[i])
    plt.imshow(img_to_show)
    plt.axis('off')
    plt.title(
        f"Real: {class_names[y_true_batch[i]]}\nPred: {class_names[y_pred_batch[i]]}
        color='green' if y_true_batch[i] == y_pred_batch[i] else 'red'
    )
plt.tight_layout()
plt.suptitle("Muestreo de Predicciones vs. Etiquetas Reales", fontsize=16, y=1.05)
plt.show()

```

1/1 ————— 0s 78ms/step

Muestreo de Predicciones vs. Etiquetas Reales

Real: Cassava Mosaic Disease (CMD)
 Pred: Cassava Mosaic Disease (CMD)



```

In [136... from sklearn.metrics import roc_curve, auc
from tensorflow.keras.utils import to_categorical

# 3. Convertir etiquetas reales a one-hot
n_classes = y_pred_probs.shape[1]
y_true_onehot = to_categorical(y_true, num_classes=n_classes)

# 4. Calcular ROC y AUC por clase
fpr = dict()
tpr = dict()
roc_auc = dict()

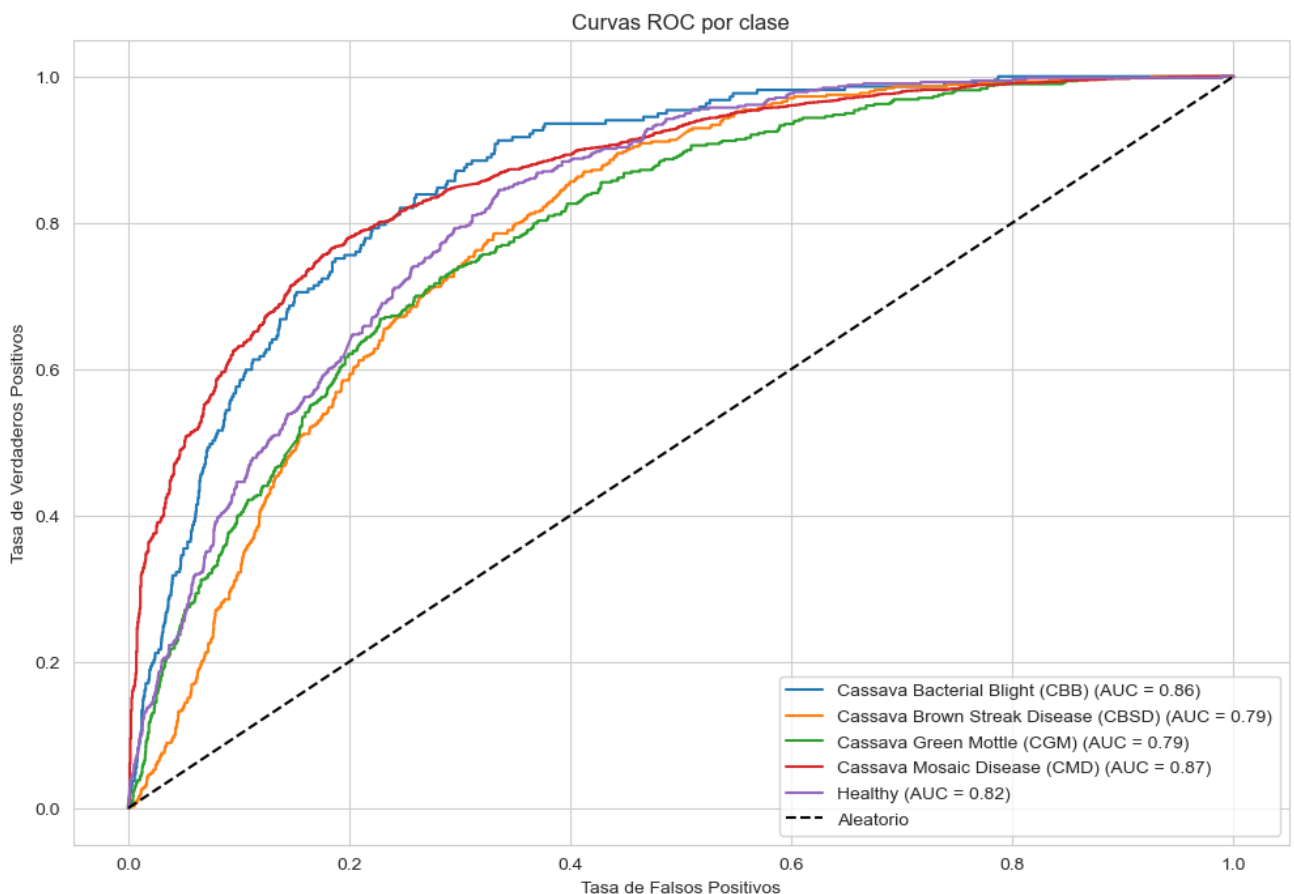
for i in range(n_classes):
    fpr[i], tpr[i], _ = roc_curve(y_true_onehot[:, i], y_pred_probs[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

```

```
# 5. Obtener nombres de clase
class_names = list(test_generator.class_indices.keys())

# 6. Graficar curvas ROC
plt.figure(figsize=(12, 8))
for i in range(n_classes):
    plt.plot(fpr[i], tpr[i], label=f'{class_names[i]} (AUC = {roc_auc[i]:.2f})')

plt.plot([0, 1], [0, 1], 'k--', label='Aleatorio')
plt.xlabel('Tasa de Falsos Positivos')
plt.ylabel('Tasa de Verdaderos Positivos')
plt.title('Curvas ROC por clase')
plt.legend(loc='lower right')
plt.grid(True)
plt.show()
```



```
In [137... from sklearn.metrics import roc_auc_score

# AUC macro (promedio de clases, todas igual de importantes)
auc_macro = roc_auc_score(y_true_onehot, y_pred_probs, average='macro')

# AUC micro (promedia sobre todas las instancias)
auc_micro = roc_auc_score(y_true_onehot, y_pred_probs, average='micro')
```

```
print(f"AUC Macro promedio: {auc_macro:.4f}")
print(f"AUC Micro promedio: {auc_micro:.4f}")
```

AUC Macro promedio: 0.8258
AUC Micro promedio: 0.8874

```
In [138]: # Asegúrate de tener esto:
# y_pred = np.argmax(y_pred_probs, axis=1)
# y_true = test_generator.classes

# Accuracy
accuracy = accuracy_score(y_true, y_pred)

# F1 Scores
f1_macro = f1_score(y_true, y_pred, average='macro')
f1_micro = f1_score(y_true, y_pred, average='micro')
f1_weighted = f1_score(y_true, y_pred, average='weighted')

print(f"Accuracy: {accuracy:.4f}")
print(f"F1 Score (macro): {f1_macro:.4f}")
print(f"F1 Score (micro): {f1_micro:.4f}")
print(f"F1 Score (weighted): {f1_weighted:.4f}")

# Detalle por clase
class_names = list(test_generator.class_indices.keys())
print("\nReporte de clasificación por clase:")
print(classification_report(y_true, y_pred, target_names=class_names))
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[138], line 9
      6 accuracy = accuracy_score(y_true, y_pred)
      8 # F1 Scores
---->  9 f1_macro = f1_score(y_true, y_pred, average='macro')
     10 f1_micro = f1_score(y_true, y_pred, average='micro')
     11 f1_weighted = f1_score(y_true, y_pred, average='weighted')

NameError: name 'f1_score' is not defined
```

```
In [ ]: fin = datetime.datetime.now()
print(f"Fin: {fin.strftime('%Y-%m-%d %H:%M:%S.%f')}")

duracion = fin - inicio
print(f"Duración: {duracion}")
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```